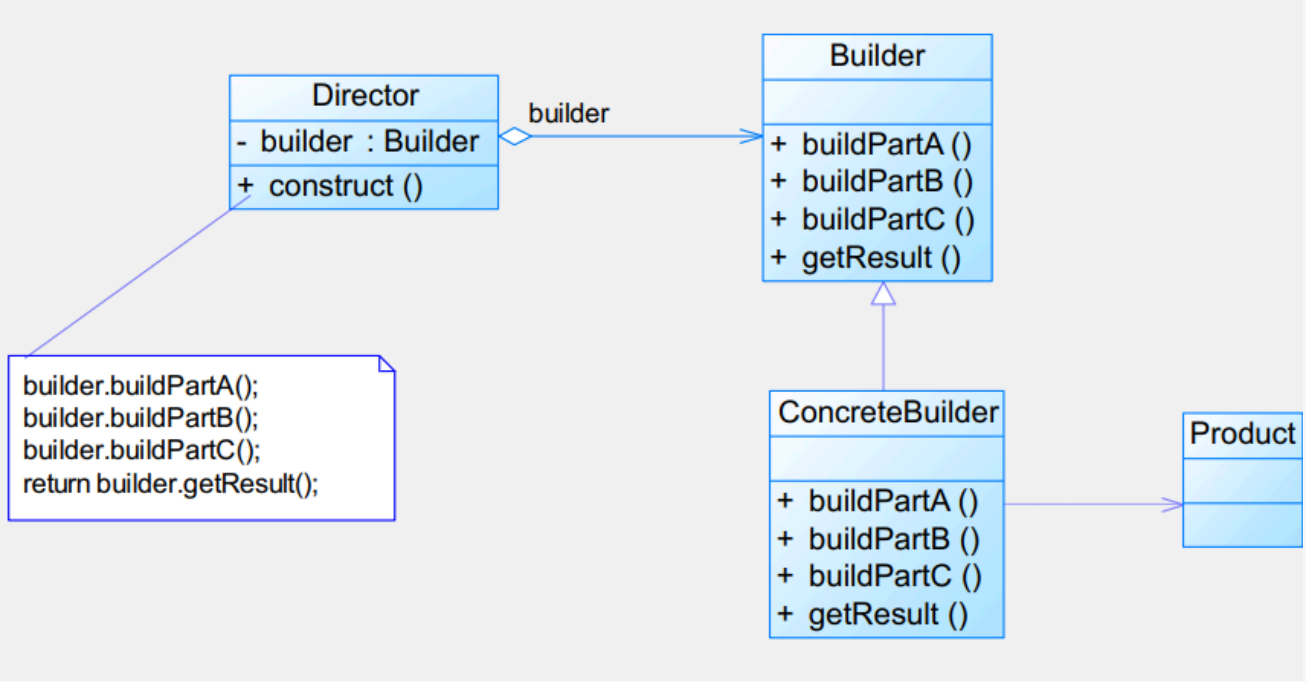
建造者模式Builder Pattern

1. 对于一个创造比较复杂的对象，其中可能需要分成很多不同的部分来进行创造，创造过程可能还有先后顺序，参数很多的情况下，如果在一个类里面完成可能会导致构造函数参数爆炸
2. 分成三个角色product builder concretebuilder director




```java
public abstract class Builder{
    protected Product product = new Product();
    public abstract void buildPartA();
    public abstract void buildPartB();
    public abstract void buildPartC();
    public Product getResult(){
        return product;
    }
}

public class ConcreteBuilder extends Builder{
     public void buildPartA(){
         product.buildA()...
     }
    public void buildPartB();
```

```
    public void buildPartC();
    public Product getResult(){
        return product;
    }
}

public class Director {
    private Builder builder;
    public Director(Builder builder){
        this.builder = builder;
    }
    public void setBuilder(Builder builder){
        this.builder = builder;
    }
    public Product construct(){
        builder.buildPartA();
        builder.buildPartB();
        builder.buildPartC();
        return builder.getResult();
    }
}
```

```
Builder builder = new ConcreteBuilder();
Director director = new Director(builder);
Product = director.construct();
```

产品之间的差异性比较大，不适合用建造者模式

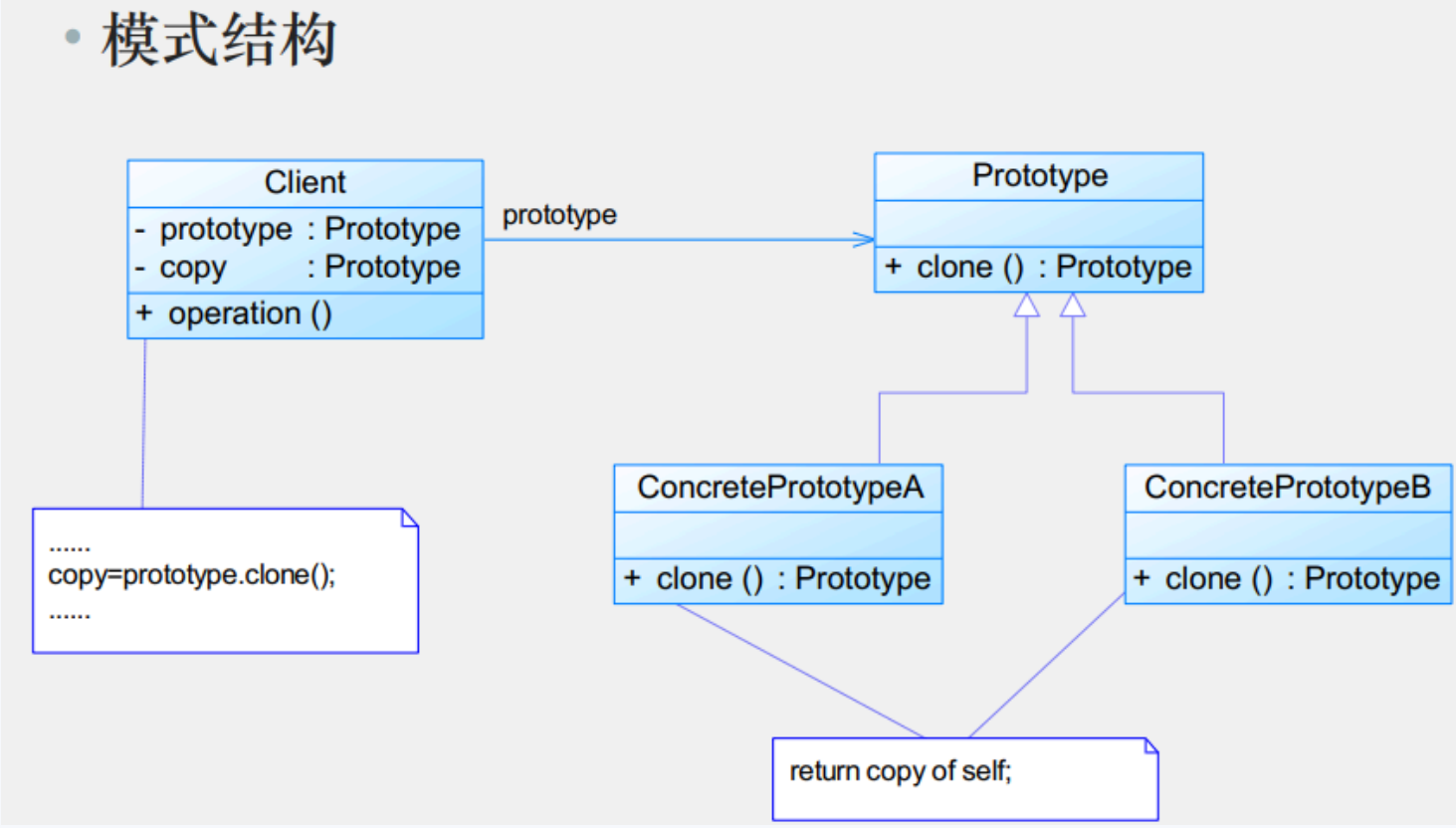
```
HttpRequest req = HttpRequest.builder()
  .url("https://example.com")
  .method("POST")
  .header("Content-Type", "application/json")
  .timeoutMs(3000)
  .body("{...}")
  .build();
```

这个是链式调用，返回的是this对象，然后接着调用下一个方法

原型模式 Prototype Pattern

注重区分浅拷贝和深拷贝就行了

核心就是因为创建一个对象的成本太高了，比如参数很多，不想再写一遍，所以就是需要多加一个clone方法，然后创建一个新的对象的时候就可以用现有的对象直接克隆就行了，如果有微调具体改那么一两个参数就可以了




```java
import java.util.ArrayList;
import java.util.List;

interface Prototype<T> {
  T clonePrototype();
}

class ConcretePrototypeA implements Prototype<ConcretePrototypeA> {
  private String name;
  private int value;
  private List<String> tags;
```

```java

  public ConcretePrototypeA(String name, int value, List<String> tags) {
    this.name = name;
    this.value = value;
    this.tags = tags;
  }

  @Override
  public ConcretePrototypeA clonePrototype() {
    List<String> tagsCopy = new ArrayList<>(this.tags);
    return new ConcretePrototypeA(this.name, this.value, tagsCopy);
  }

  public void setName(String name) { this.name = name; }
  public void setValue(int value) { this.value = value; }
  public void addTag(String tag) { this.tags.add(tag); }

  @Override
  public String toString() {
    return "ConcretePrototypeA{name='" + name + "', value=" + value + ", tags=" + tags + "}";
  }
}

class ConcretePrototypeB implements Prototype<ConcretePrototypeB> {
  private String type;
  private double score;

  public ConcretePrototypeB(String type, double score) {
    this.type = type;
    this.score = score;
  }

  @Override
  public ConcretePrototypeB clonePrototype() {
    return new ConcretePrototypeB(this.type, this.score);
```

```java
  }

  public void setType(String type) { this.type = type; }
  public void setScore(double score) { this.score = score; }

  @Override
  public String toString() {
    return "ConcretePrototypeB{type='" + type + "', score=" + score + "}";
  }
}

public class PrototypePatternDemo {
  public static void main(String[] args) {
    ConcretePrototypeA prototypeA = new ConcretePrototypeA(
        "A-Template",
        100,
        new ArrayList<>(List.of("default"))
    );

    ConcretePrototypeA copyA = prototypeA.clonePrototype();
    copyA.setName("A-Copy");
    copyA.addTag("custom");

    ConcretePrototypeB prototypeB = new ConcretePrototypeB("B-Template", 0.8);
    ConcretePrototypeB copyB = prototypeB.clonePrototype();
    copyB.setScore(0.95);

    System.out.println(prototypeA);
    System.out.println(copyA);
    System.out.println(prototypeB);
    System.out.println(copyB);
  }
}
```

关于接口和继承的实现

继承

```
class Animal {
    protected String name;

    public Animal(String name){
        this.name = name;
    }
    public void speak(){
        System.out.println("...");
    }
}

class Dog extends Animal{
    public Dog(String name){
        super(name);
    }
    @Override
    public void speak(){
        System.out.println(name+"says wang wang");
    }
}

Animal a = new Dog("Bob");
a.speak();
```

```
interface Flyable{
    void fly();//默认public abstract
}
interface RunnableFast{
    void runFast();
}
Class Bird implements Flyable,RunnableFast{
    @Override
```

```
    public void fly(){
        System.out.println("Bird Flying");
    }
    @Override
    public void runFast(){
        System.out.println("Bird running");
    }
}

Flyable f = new Bird();
f.fly();

```